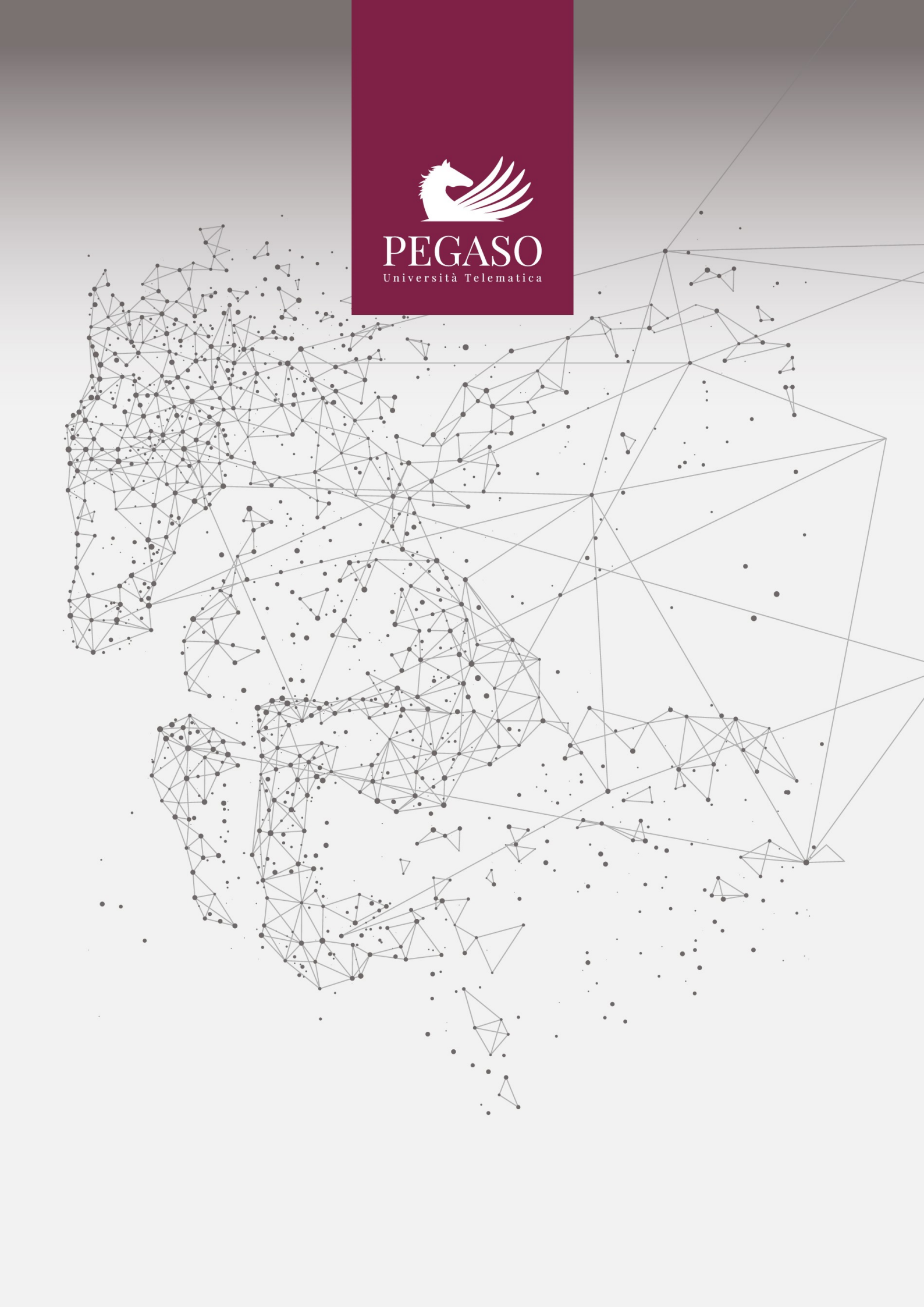

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. REGOLA PER LA RAPPRESENTAZIONE DELL'OPPOSTO DI UN NUMERO 3
2. SOTTRAZIONE COME SOMMA CON L'OPPOSTO 6
3. REGOLA PER RICONOSCERE L'OVERFLOW 8
4. REGOLA PER L'ESTENSIONE DEL SEGNO 12
BIBLIOGRAFIA 14

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Regola per la rappresentazione dell'opposto di un numero

La *Notazione in complemento a* 2, (detta più in generale "Notazione in complemento alla base B", nel nostro caso B=2) è il metodo più diffuso per la rappresentazione dei numeri interi all'interno del computer. La sua diffusione è dovuta sia alla possibilità di eseguire l'addizione dei numeri interi senza dover esaminare preliminarmente la relazione tra i valori dei moduli degli operandi, sia a proprietà relative alla rappresentazione dell'opposto di un numero ed al riconoscimento del verificarsi dell'errore di Overflow.

Una prima proprietà stabilisce una regola semplice e veloce per ottenere la rappresentazione dell'opposto di un numero a partire dalla rappresentazione del numero dato.

Questo consente di realizzare l'Hardware per l'esecuzione della sottrazione utilizzando l'Hardware che realizza l'addizione con piccole modifiche che permettono l'esecuzione di entrambe le operazioni.

Notazione in complemento a 2

REGOLA per ottenere la rappresentazione dell'opposto:

- complementare la sequenza bit a bit (cambiare ogni 0 in 1 e ogni 1 in 0);
- sommare il valore 1.

La validità della regola per ottenere la rappresentazione dell'opposto può essere formalmente dimostrata.

Qui di seguito vengono mostrati esempi di applicazione della regola con sequenze binarie di diverse lunghezze.

ESEMPIO

Regola per la rappresentazione dell'opposto di un numero

1 0 0 1 1 1 ⟷ −25 numero dato

0 0 0 0 0 0 0	*riporti*
0 1 1 0 0 0 +	complemento bit a bit
0 0 0 0 0 1 =	aggiunta del valore 1
0 1 1 0 0 1 ⟷ (+25)	opposto

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

ESEMPIO

Regola per la rappresentazione dell'opposto di un numero

0 0 1 0 0 1 0 1 ⟷ +37 numero dato

complemento bit a bit più 1

0 0 0 0 0 0 0 0 0 *riporti*

1 1 0 1 1 0 1 0 + complemento bit a bit

0 0 0 0 0 0 0 1 = aggiunta del valore 1

1 1 0 1 1 0 1 1 ⟷ −37 opposto

ESEMPIO

Regola per la rappresentazione dell'opposto di un numero

1 1 0 0 ⟷ −4

complemento bit a bit più 1

0 0 1 1 0 *riporti*

0 0 1 1 + complemento bit a bit

0 0 0 1 = aggiunta del valore 1

0 1 0 0 ⟷ +4 opposto

I registri del processore MIPS possono memorizzare sequenze di 32 bit. Qui di seguito viene dato un esempio di applicazione della regola per sequenze di tale lunghezza.

ESEMPIO

Regola per la rappresentazione dell'opposto di un numero

1 1 1 1 1 0 0 0 0 0 1 1 1 1 1 1 1 1 1 0 0 0 1 1 1 1 1 1 0 0 0 numero

complemento bit a bit più 1

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 *riporti*

0 0 0 0 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 0 1 1 1 +

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 =

0 0 0 0 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 1 0 0 0 opposto

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per stabilire la correttezza della rappresentazione ottenuta si può effettuare la somma del numero con l'opposto e verificare che si ottiene 0 come risultato:

1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 riporti

1 1 1 1 1 0 0 0 0 1 1 1 1 1 1 1 1 1 0 0 0 1 1 1 1 1 1 0 0 0 + numero

0 0 0 0 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 1 0 0 0 = opposto

---

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Sottrazione come somma con l'opposto

L'esecuzione dell'operazione di sottrazione utilizzando la Notazione in modulo e segno, che rappresenta il segno separato dal modulo, pone il problema di dover effettuare il controllo sui moduli e sui segni degli operandi prima di poter procedere ai calcoli. Questo determina un maggior tempo di esecuzione e una maggiore complessità dell'Harware.

Il grande vantaggio offerto dalla notazione in complemento a 2 sta nel fatto che la semplice regola per ottenere la rappresentazione dell'opposto di un numero consente di sfruttare la relazione matematica che definisce la sottrazione come somma con l'opposto:

$$A - B = A + (-B)$$

VANTAGGIO
si può effettuare la SOTTRAZIONE utilizzando lo stesso Hardware che effettua l'ADDIZIONE aggiungendo piccole modifiche che forniscono la rappresentazione dell'opposto del numero da sottrarre.

Nella realizzazione dell'Unità Aritmetico-Logica del MIPS viene utilizzata questa proprietà per realizzare l'esecuzione della sottrazione con piccole modifiche dell'Hardware che effettua l'addizione.

ESEMPIO

Sottrazione utilizzando la relazione $A - B = A + (-B)$

$16 - 21 = 16 + (-21) = -5$

$+16 \longleftrightarrow 0\ 1\ 0\ 0\ 0\ 0$

$+21 \longleftrightarrow 0\ 1\ 0\ 1\ 0\ 1$ numero di cui determinare l'opposto

calcolo dell'opposto:

riporti	*0 0 0 0 0 0 0*	
	1 0 1 0 1 0 +	complemento bit a bit
	0 0 0 0 0 1 =	somma del valore 1
$-21 \longleftrightarrow$	1 0 1 0 1 1	opposto

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

addizione con l'opposto:

riporti 0 0 0 0 0 0 0

+16 0 1 0 0 0 0 +

–21 1 0 1 0 1 1 =

–5 ⟷ 1 1 1 0 1 1

Le dichiarazioni di tipo poste all'interno di un programma in linguaggio ad Alto Livello segnalano al Compilatore quale Notazione bisogna utilizzare per la rappresentazione del valore della variabile.

Per il Linguaggio C la dichiarazione di tipo

int K ;

indica che la variabile K è un numero intero il cui valore deve essere rappresentato all'interno della macchina utilizzando la Notazione in complemento a 2.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Regola per riconoscere l'Overflow

Poiché la Notazione in complemento a 2, come tutte le notazioni, utilizza sequenze di lunghezza finita su un alfabeto finito, è possibile rappresentare solo un insieme finito di valori compresi tra un MASSIMO ed un MINIMO. Ricordiamo che l'intervallo dei valori rappresentabili con sequenze di lunghezza N è individuato da due estremi espressi in funzione di tale valore:

Sequenze di lunghezza N

INTERVALLO DI RAPPRESENTAZIONE = $[-2^{N-1}, 2^{N-1} - 1]$

Quando il risultato di un'operazione produce un valore fuori dell'intervallo di rappresentazione si verifica l'errore di OVERFLOW, e la sequenza che rappresenta il risultato corrisponde ad un valore diverso da quello corretto.

Il termine inglese *overflow* significa *traboccamento* ed è usato perché l'errore è determinato dal fatto che la rappresentazione corretta del risultato richiederebbe un'ulteriore bit da aggiungere a sinistra della sequenza per rappresentare una cifra del risultato, che altrimenti viene persa a causa della lunghezza insufficiente (cioè, avviene un "traboccamento" della cifra fuori dalla sequenza).

L'errore di OVERFLOW rappresenta un problema per i calcoli effettuati da un computer in quanto, se non viene rilevato, porta a dei risultati completamente sbagliati che possono essere, erroneamente, ritenuti corretti.

In generale è possibile stabilire il seguente risultato in relazione ai casi in cui si può verificare un errore di Overflow per effetto dell'addizione di due numeri interi:

Addizione tra operandi con segno

- **Segni diversi**: MAI OVERFLOW

il modulo dal risultato è sempre minore del modulo di entrambi gli operandi perché i moduli degli operandi si sottraggono.

- **Segni uguali: POSSIBILE OVERFLOW**

il modulo dal risultato è maggiore del modulo di entrambi gli operandi perché i moduli degli operandi si sommano.

Qui di seguito vengono mostrati alcuni esempi di addizioni che determinano un errore di Overflow allo scopo di evidenziare una caratteristica comune che coinvolge gli ultimi due riporti calcolati. Si può dimostrare che questa caratteristica fornisce una REGOLA generale per il riconoscimento del verificarsi dell'errore di Overflow nella Notazione in complemento a 2.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

ESEMPIO

Lunghezza delle sequenze N = 6 bit**.**

Intervallo di valori rappresentabili:

$[-2^{N-1}, 2^{N-1}-1] = [-2^{6-1}, 2^{6-1}-1] = [-32, 31]$

addendi:

–16 ⟷ 1 1 0 0 0 0

–30 ⟷ 1 0 0 0 1 0

somma:

–16 + (–30) = – 46 fuori dall'intervallo ⟹ OVERFLOW.

Verifichiamo che, a causa dell'Overflow, si ottiene un risultato sbagliato effettuando l'addizione delle sequenze binarie. Nel calcolo viene evidenziato il fatto che gli ultimi due riporti calcolati hanno valori diversi:

riporti *1 0* *0 0 0 0* **ultimi riporti diversi**

1 1 0 0 0 0 +

1 0 0 0 1 0 =

+18 ⟷ 0 1 0 0 1 0 **ERRORE**.

ESEMPIO

Lunghezza delle sequenze N = 5 bit.

Intervallo di valori rappresentabili:

$[-2^{N-1}, 2^{N-1}-1] = [-2^{5-1}, 2^{5-1}-1] = [-16, 15]$

addendi:

+9 0 1 0 0 1

+14 0 1 1 1 0

somma:

+ 9 + 14 = + 23 fuori dall'intervallo ⟹ OVERFLOW.

Verifica:

riporti *0 1* *0 0 0 0* **ultimi riporti diversi**

0 1 0 0 1 +

0 1 1 1 0 =

– 9 ⟷ 1 0 1 1 1 **ERRORE**.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Si può dimostrare che la relazione che si osserva sugli ultimi due riporti calcolati in questi due esempi particolari caratterizza il verificarsi di un Overflow, cioè, se tali riporti sono uguali il risultato è corretto, se sono diversi in risultato è sbagliato a causa dell'Overflow.

Diamo, quindi la seguente formulazione generale di tale proprietà che viene utilizzata nella realizzazione dell'Unità Aritmetico-Logica per produrre un segnale che consente di riconoscere il verificarsi dell'Overflow:

REGOLA per riconoscere l'Overflow

Indicando con: C_{N-1} il riporto sul bit più significativo

C_N il riporto non utilizzato

quando questi riporti sono DIVERSI si ha un errore di OVERFLOW:

$$C_N \neq C_{N-1}$$

quando i due riporti sono uguali il risultato è corretto.

Il seguente esempio mostra un caso di Overflow per operandi memorizzati nei registri del processore MIPS. In questo caso, poiché la lunghezza delle sequenze è di 32 bit, la condizione che segnala l'avvenuto Overflow è espressa dalla relazione

$$c_{32} \neq c_{31}$$

esemplificata nel seguente calcolo:

riporti	*01100000000000000000000000000000*
primo operando	01111000000000000000000000000110 +
secondo operando	01100000000000000000000000000001 =
risultato	11010000000000000000000000000111

Anche senza calcolare i valori dei numeri rappresentati in complemento a due si può affermare che il risultato è sbagliato osservando il suo bit di segno, perché si è ottenuto un valore negativo sommando operandi entrambi positivi.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Infine, osserviamo che data una lunghezza N, l'opposto positivo del minimo numero negativo -2^{N-1} non è compreso nell'intervallo dei numeri rappresentabili in complemento a 2 con sequenze di N bit, quindi l'applicazione della regola per determinare la rappresentazione dell'opposto in tal caso produce un risultato sbagliato, comunque segnalato dal verificarsi della condizione di Overflow sui riporti:

ESEMPIO

Lunghezza delle sequenze N = 4 bit.

Intervallo di valori rappresentabili:

$[-2^{N-1}, 2^{N-1} - 1] = [-2^{4-1}, 2^{4-1} - 1] = [-8, 7]$

Regola per la rappresentazione dell'opposto di un numero

1 0 0 0 ⟷ (−8)

complemento bit a bit più 1

0 1 1 1 0	*riporti*
0 1 1 1 +	complemento bit a bit
0 0 0 1 =	aggiunta del valore 1
1 0 0 0 ⟷ (−8)	ERRORE di Overflow

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Regola per l'estensione del segno

La notazione in complemento a 2 consente, con una semplice regola, di aumentare il numero di bit della rappresentazione di un numero senza modificare il valore rappresentato:

REGOLA della estensione del segno

Aggiungendo a sinistra della sequenza un qualunque numero di bit tutti uguali al bit di segno il valore rappresentato non cambia.

Anche la validità di questa regola può essere dimostrata in generale.

Qui di seguito vengono mostrati due esempi di applicazione per un numero positivo e negativo verificando l'invarianza dei valori rappresentati.

ESEMPIO

Verifica dell'estensione del segno per numeri negativi.

Numero dato:

$1\ 1\ 0\ 1 \quad = -1x2^3 + 1x2^2 + 0x2^1 + 1x2^0 =$

$= -8 + 4 + 0 + 1 = -8 + 5 = -3$

Passando dalla lunghezza N = 4 alla lunghezza N = 6 aggiungendo due bit a sinistra uguali al valore 1 del bit di segno si ha:

$1\ 1\ 1\ 1\ 0\ 1 \quad = 1x(-2^5) + 1x2^4 + 1x2^3 + 1x2^2 + 0x2^1 + 1x2^0 =$

$= -32 + 16 + 8 + 4 + 0 + 1 = -32 + 29 = -3$

Il valore rappresentato non è cambiato.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

ESEMPIO

Verifica dell'estensione del segno per numeri positivi:

Numero dato:

$0\ 1\ 0\ 1 \quad = -0x2^3 + 1x2^2 + 0x2^1 + 1x2^0 =$

$= -0 + 4 + 0 + 1 =$ +5

Passando dalla lunghezza N = 4 alla lunghezza N = 7 aggiungendo tre bit a sinistra uguali al valore 0 del bit di segno si ha:

$0\ 0\ 0\ 0\ 1\ 0\ 1 \quad = 0x(-2^6) + 0x2^5 + 0x2^4 + 0x2^3 + 1x2^2 + 0x2^1 + 1x2^0 =$

$= -0 + 0 + 0 + 0 + 0 + 4 + 0 + 1 =$ +5

Il valore rappresentato non è cambiato.

La regola dell'estensione del segno viene utilizzata nel progetto dell'Architettura MIPS per calcolare l'indirizzo dei dati memorizzati nella Memoria principale nella esecuzione delle istruzioni di trasferimento.

Le proprietà della Notazione in complemento a 2 consentono di semplificare l'Hardware del computer realizzando un dispositivo per eseguire l'addizione che, con piccole integrazioni, permette di effettuare anche la sottrazione mediante la determinazione dell'opposto del numero da sottrarre, e di segnalare situazioni di Overflow.

Nel corso degli anni le Notazioni da adottare per la rappresentazione dei numeri sono state ampiamente regolamentate da *standard* emanati da organismi internazionali. La notazione in complemento a 2 per gli interi relativi è la soluzione comunemente adottata nella progettazione dell'Unità Aritmetico-Logica (ALU).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- D. A. Patterson, J. L. Hennessy, "Struttura e progetto dei calcolatori", Zanichelli.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).